```
    http.Handle("/metrics", promhttp.HandlerFor(
        registry,
        promhttp.HandlerOpts{
            EnableOpenMetrics: true,
        },
    ))
    tp, err := tracerProvider("http://localhost:14268/api/traces")
    if err != nil {
        log.Fatal(err)
    }
    otel.SetTracerProvider(tp)
    log.Fatal(http.ListenAndServe(":4242", nil))
}
```

Sets up the OpenTelemetry config for Jaeger as the trace provider

Now, let's start the service and generate some spans.

2.5.2 Telemetry

We can now execute the `echo` service locally and ingest traces like so:

```
$ ./echosvc
```

We can ingest the traces of our `echo` service using Jaeger (https://www.jaegertracing.io/), which we then use as the data source for Grafana.

To set up the destination, use the Docker Compose manifest file shown in the following listing to set up Jaeger and Grafana.

Listing 2.11 Using docker-compose.yaml to stand up Jaeger and Grafana

```
version: "3.5"
networks:
  jaeger:
services:
  jaeger:
    image: jaegertracing/all-in-one:latest
    ports:
    - "6831:6831/udp"
    - "16686:16686"
    - "14268:14268"
    networks:
    - jaeger
  grafana:
    image: grafana/grafana:latest
    ports:
    - "3000:3000"
    networks:
    - jaeger
```